



UNIVERSIDADE FEDERAL
DE MATO GROSSO

Geração de Código Objeto

Raphael de Souza – raphael@ic.ufmt.br

Código Objeto

- ▶ Pode ser:
 - ▶ Assembly
 - ▶ Pseudo código de uma máquina hipotética

Código Objeto

- ▶ É necessário conhecer as características da máquina-alvo
- ▶ Nossa máquina hipotética será chamada de **MaqHipo**

Código Objeto

- ▶ MaqHipo será uma máquina à pilha

Código Objeto

- ▶ MaqHipo terá uma memória composta de 3 áreas:
 - ▶ Área de código
 - ▶ Vetor que conterà as instruções geradas pelo compilador
 - ▶ Área de Dados
 - ▶ Pilha que conterà os valores manipulados pelas instruções da MaqHipo
 - ▶ Área dos Registradores de base
 - ▶ Pilha que armazena dados sobre o escopo
- ▶ Supõe-se que cada área tem endereços numerados com 0, 1, 2, ...

Código Objeto

- ▶ Área de Código
 - ▶ Simulada como um array (chamaremos de C)
 - ▶ Cada registro do array é uma instrução ou pseudo-código
 - ▶ a Geração de Código consiste no preenchimento desse array, que será "interpretado" posteriormente

Código Objeto

- ▶ Área de Dados
 - ▶ Simulada por uma pilha (chamaremos de D)
 - ▶ Armazena os valores manipulados pelas instruções
 - ▶ Só existirá realmente durante a interpretação do código gerado
- ▶ Toda instrução agirá sobre o topo desta pilha ou sobre o topo e seu antecessor (no caso de operações binárias)

Código Objeto

- ▶ Área dos Registradores de Base
 - ▶ Pilha de registradores (chamaremos de B)
 - ▶ Cada registrador, quando ativo, aponta para um índice de **D** que marca o início de um novo escopo
 - ▶ Não será necessário pois nossa linguagem não permite aninhamento de procedimentos

Código Objeto

- ▶ Além das 3 áreas, a MaqHipo possui registradores especiais:
- ▶ o registrador de programa **i** aponta para próxima instrução a ser executada, portanto **C(i)**
- ▶ o registrador **s** indica o topo da pilha D, portanto **D(s)**
- ▶ o registrador **b** indica o topo da pilha B, portanto **B(b)**

Código Objeto

- ▶ Como a área D só manipula dados e LALG só manipula inteiros e reais, então:
 - ▶ D: array de real
 - ▶ s: topo da pilha
- ▶ A Tabela de Símbolos – TS – deverá ser aumentada com 2 novos campos: **end_rel** (índice na pilha D) e **prim_instr** (endereço da 1ª instrução no array C, no caso do identificador ser do tipo procedimento).

Código Objeto

- ▶ Veremos agora o conjunto de instrução da MaqHipo

Código Objeto

- ▶ CRCT k
 - ▶ Carrega constante k no topo da pilha D
 - ▶ $s := s + 1$;
 - ▶ $D[s] := k$
- ▶ CRVL n
 - ▶ Carrega valor de endereço n no topo da pilha D
 - ▶ $s := s + 1$;
 - ▶ $D[s] := D[n]$

Código Objeto

▶ SOMA

- ▶ Soma o elemento antecessor com o topo da pilha; desempilha os dois e empilha o resultado
- ▶ $D[s-1] := D[s-1] + D[s];$
- ▶ $s := s - 1$

▶ SUBT

- ▶ Subtrai o antecessor pelo elemento do topo
- ▶ $D[s-1] := D[s-1] - D[s];$
- ▶ $s := s - 1$

Código Objeto

▶ MULT

- ▶ Multiplica elemento antecessor pelo elemento do topo
- ▶ $D[s-1] := D[s-1] * D[s];$
- ▶ $s := s - 1$

▶ DIVI

- ▶ Divide o elemento antecessor pelo elemento do topo
- ▶ $D[s-1] := D[s-1] \text{ div } D[s];$
- ▶ $s := s - 1$

Código Objeto

▶ INVE

- ▶ Inverte sinal do topo
- ▶ $D[s] := -D[s]$

▶ CONJ

- ▶ Conjunção de valores lógicos. $F=0; V=1$
- ▶ se $D[s-1] = 1$ e $D[s] = 1$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

Código Objeto

▶ DISJ

- ▶ Disjunção de valores lógicos
- ▶ se $D[s-1] = 1$ ou $D[s] = 1$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

▶ NEGA

- ▶ Negação lógica
- ▶ $D[s] := 1 - D[s]$

Código Objeto

▶ CPME

- ▶ Comparação de menor entre o antecessor e o topo
- ▶ se $D[s-1] < D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

▶ CPMA

- ▶ Comparação de maior
- ▶ se $D[s-1] > D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

Código Objeto

▶ CFIG

- ▶ Comparação de igualdade
- ▶ se $D[s-1] = D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

▶ CDES

- ▶ Comparação de desigualdade
- ▶ se $D[s-1] \neq D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

Código Objeto

▶ CPMI

- ▶ Comparação menor-igual
- ▶ se $D[s-1] \leq D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

▶ CMAI

- ▶ Comparação maior-igual
- ▶ se $D[s-1] \geq D[s]$ então $D[s-1] := 1$
senão $D[s-1] := 0$;
- ▶ $s := s - 1$

Código Objeto

- ▶ ARMZ n
 - ▶ Armazena o topo da pilha no endereço n de D
 - ▶ $D[n] := D[s];$
 - ▶ $s := s - 1$
- ▶ ex: $a := a + 1$
 - CRVL a^*
 - CRCT 1
 - SOMA
 - ARMZ a^*

Código Objeto

- ▶ DSVI p
 - ▶ Desvio incondicional para a instrução de endereço p
 - ▶ $i := p$
- ▶ DSVF p
 - ▶ Desvio condicional para a instrução de endereço p; o desvio será executado caso a condição resultante seja falsa; o valor da condição estará no topo
 - ▶ se $D[s]=0$ então $i := p$
 - ▶ $s := s - 1$

Código Objeto

► Ex:

if $x > I$ then

$x := z$

else

$y := I$

CRVL x^*

CRCT I

CPMA

DSVF 'número instrução do else'

CRVL z^*

ARMZ x^*

DSVI 'número instrução depois do if'

CRCT I

ARMZ y^*

Código Objeto

▶ LEIT

- ▶ Lê um dado de entrada para o topo da pilha
- ▶ $s := s + 1$;
- ▶ $D[s] := \text{"valor da entrada"}$

▶ IMPR

- ▶ Imprime valor o valor do topo da pilha na saída
- ▶ "imprimir $D[s]$ ";
- ▶ $s := s - 1$

Código Objeto

- ▶ ALME m
 - ▶ Reserva m posições na pilha D; m depende do tipo da variável
 - ▶ $s := s + m$
- ▶ Ex
 - ▶ var a, b: integer
 - ALME 1
 - ALME 1
 - ▶ Ao mesmo tempo, na TS, coloca-se no campo de end_rel de a e b, os valores s+1 e s+2 (0 e 1), respectivamente (pois são as primeiras variáveis declaradas)

Código Objeto

- ▶ INPP

- ▶ Inicia programa – será sempre a 1ª instrução

- ▶ $s := -1$

- ▶ PARA

- ▶ Termina a execução do programa

Código Objeto

<u>public class</u> <u>Teste</u> {	0. <u>INPP</u>	<u>c = a - b;</u>	19. <u>CRVL 1</u>
<u>public static void</u> <u>main</u> (<u>String[]</u> <u>args</u>) {			20. <u>CRVL 2</u>
<u>double</u> <u>cont</u> ;	1. <u>ALME 1</u>		21. <u>SUBT</u>
<u>double</u> <u>a,b,c</u> ;	2. <u>ALME 1</u>		22. <u>ARMZ 3</u>
	3. <u>ALME 1</u>	<u>} else {</u>	23. <u>DSVI 28</u>
	4. <u>ALME 1</u>	<u>c = b - a;</u>	24. <u>CRVL 2</u>
<u>cont = 10;</u>	5. <u>CRCT 10</u>		25. <u>CRVL 1</u>
	6. <u>ARMZ 0</u>		26. <u>SUBT</u>
<u>while</u> (<u>cont > 0</u>) {	7. <u>CRVL 0</u>		27. <u>ARMZ 3</u>
	8. <u>CRCT 0</u>	<u>}</u>	
	9. <u>CPMA</u>	<u>System.out.println</u> (<u>c</u>);	28. <u>CRVL 3</u>
	10. <u>DSVF 35</u>		29. <u>IMPR</u>
<u>a = lerDouble</u> ();	11. <u>LEIT</u>	<u>cont = cont - 1;</u>	30. <u>CRVL 0</u>
	12. <u>ARMZ 1</u>		31. <u>CRCT 1</u>
<u>b = lerDouble</u> ();	13. <u>LEIT</u>		32. <u>SUBT</u>
	14. <u>ARMZ 2</u>		33. <u>ARMZ 0</u>
<u>if</u> (<u>a > b</u>) {	15. <u>CRVL 1</u>	<u>}</u>	34. <u>DSVI 7</u>
	16. <u>CRVL 2</u>	<u>System.out.println</u> (<u>c</u>);	35. <u>CRVL 3</u>
	17. <u>CPMA</u>		36. <u>IMPR</u>
	18. <u>DSVF 24</u>	<u>}</u>	
		<u>}</u>	37. <u>PARA</u>

Referências Bibliográficas

- ▶ Livro:

- ▶ Compiladores: Princípios, técnicas e ferramentas. Alfred V. Aho, et al